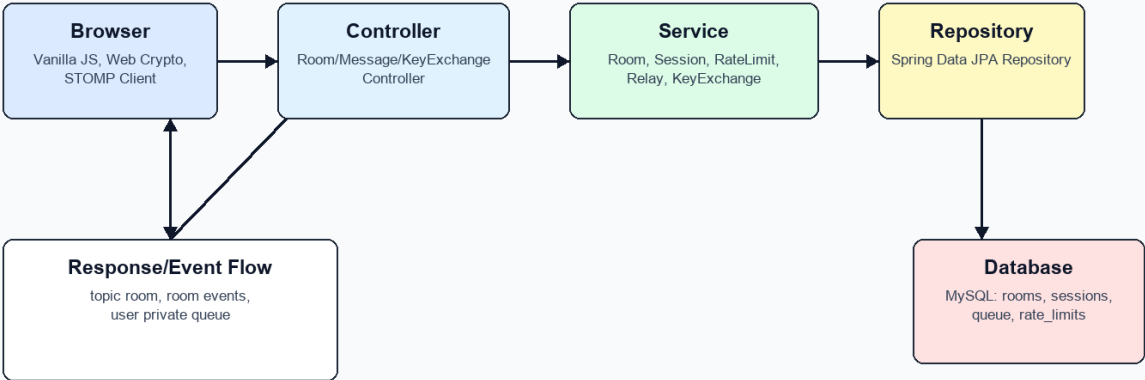


Software Architecture - Manhattan

Architecture Diagram - Manhattan



Tabel Layer

Layer	Class/File	Tanggung Jawab
Controller	RoomController, MessageController, KeyExchangeController	Menerima STOMP message dari browser dan mengirim response/event.
Service	RoomService, SessionService, RateLimitService, MessageRelayService, KeyExchangeService	Mengelola logika bisnis, seperti rate limit, relay pesan, dan key exchange.
Repository	RoomRepository, SessionRepository, QueuedMessageRepository, RateLimitRepository	Mengakses dan menyimpan data ke database.
Entity	Room, Session, QueuedMessage, RateLimit, SessionInfo	Representasi tabel database dan status domain.
DTO	RoomCreationResult, RoomJoinResult, RoomInfo, RoomInfoResponse	Payload operasi yang dikirim antar layer.

Arsitektur layered dipilih karena memisahkan controller, service, repository, entity, dan DTO. Dengan pola ini, logic bisnis tidak bercampur dengan detail transport WebSocket atau akses database.

Screenshot Kode

Architecture Evidence - WebSocketConfig

```
17 public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {
18
19     private final IpGuardInterceptor ipGuardInterceptor;
20
21     public WebSocketConfig(IpGuardInterceptor ipGuardInterceptor) {
22         this.ipGuardInterceptor = ipGuardInterceptor;
23     }
24
25     @Override
26     public void configureMessageBroker(MessageBrokerRegistry config) {
27         ThreadPoolTaskScheduler taskScheduler = new ThreadPoolTaskScheduler();
28         taskScheduler.setPoolSize(1);
29         taskScheduler.setThreadNamePrefix("ws-heartbeat-");
30         taskScheduler.initialize();
31
32         config.enableSimpleBroker("/topic", "/queue")
33             .setHeartbeatValue(new long[]{25000, 10000})
34             .setTaskScheduler(taskScheduler);
35
36         config.setApplicationDestinationPrefixes("/app");
37         config.setUserDestinationPrefix("/user");
38     }
39
40     @Override
41     public void registerStompEndpoints(StompEndpointRegistry registry) {
42         // Native WebSocket endpoint (for @stomp/stompjs client)
43         registry.addEndpoint("/ws")
44             .addInterceptors(ipGuardInterceptor)
45             .setAllowedOriginPatterns("*");
46
47         // SockJS fallback endpoint (for older browsers)
48         registry.addEndpoint("/ws-sockjs")
49             .addInterceptors(ipGuardInterceptor)
50             .setAllowedOriginPatterns("*")
51             .withSockJS();
```